
Abstract xxxx

1 Introduction

xxxx

2 Background and Motivation

2.1 Data Stealing

2.2 Motivation

3 Empirical Study Setup

xxxx

Research Questions. We design our evaluation to answer the following four research questions.

- **RQ1 Effectiveness Evaluation:** How is the effectiveness of CEREbro, compared with the state-of-the-art malicious package detection approaches?
- **RQ2 Efficiency Evaluation:** How is the performance overhead of CEREbro?
- **RQ3 Dataset Scale Evaluation:** How does the scale of the dataset affect the effectiveness of CEREbro?
- **RQ4 Ablation Study:** How behavior sequence contributes to the effectiveness of CEREbro?
- **RQ5 Usefulness Evaluation:** How useful is CEREbro in real-world detection on PyPI and NPM?

3.1 Data Stealing Package Collection

3.2 Research Questions

RQ1 Prevalence Study.

RQ1 Prevalence Study.

RQ1 Prevalence Study.

RQ1 Prevalence Study.

RQ1 Prevalence Study.

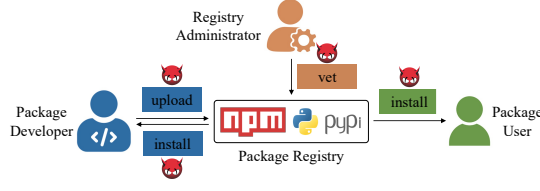


Fig. 1: Mitigations against Risky Actions and Risk Types

Table 1: Evaluation Result in Mono-Lingual Scenario for Learning-based Approaches

Train	Test	CEREBRO <i>BERT</i>		CEREBRO <i>RoBERTa</i>		CEREBRO <i>T5</i>		AMALFI _{DT}		AMALFI _{NB}		AMALFI _{SVM}		SAP		MPHUNTER	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
PyPI	PyPI	95.8%	89.4%	96.0%	91.7%	93.4%	70.9%	81.9%	83.9%	59.5%	7.5%	30.3%	30.9%	83.9%	61.5%	21.8%	85.1%
NPM	NPM	98.2%	91.8%	98.5%	92.9%	98.9%	91.0%	92.7%	86.0%	92.1%	77.1%	10.2%	26.7%	90.5%	35.4%	–	–

4 Results

4.1 Prevalence Study (RQ1)

4.2 Data Stealing Pattern Study (RQ2)

4.3 Disguise Characteristic Study (RQ3)

4.4 Stealing Objective Study (RQ4)

4.5 Malware Detection Effectiveness Evaluation (RQ5)

4.6 Threats and Limitations

Threats. First, as the campaign against malware continues, new strains of malicious packages emerge daily. This implies that the original dataset of malicious packages used in our evaluation may not encompass the full spectrum of new malicious packages, hindering CEREBRO’s effectiveness. However, we have demonstrated the effectiveness of CEREBRO in learning from newly published packages and adapting to detect emerging threats. Second, while we thoroughly examine the potentially malicious package versions identified by CEREBRO, we acknowledge that the vast workload makes it impractical to validate every potentially benign package version. Therefore, we only systematically report the result of precision but not the result of recall. Third, CEREBRO can generate false positives. To address this, we conduct manual validation on identified package versions before reporting them to PyPI and NPM teams. While this process can also introduce false positives, the fact that the reported packages are subsequently removed indicates the validity of our validation. Fourth, as pre-trained language models continue to advance rapidly, we expect to see new and more powerful models. Nevertheless, the choice of language models is orthogonal to our approach. We plan to test the effectiveness of additional language models on this task. Fifth, the varying behavior sequences between

malicious and benign packages may reduce the accuracy of CEREbro in cross-lingual scenarios. Nevertheless, the cross-lingual setting represents an extreme case in real-world where there are no malicious samples within the same ecosystem.

Limitations. First, our method is designed for PyPI and NPM packages. However, our method can be easily generalized into other ecosystems by implementing the feature extractor and behavior sequence generator for a new ecosystem. The feature set is language-agnostic. By leveraging the existing AST parser and call graph generator of the new ecosystem, we believe that the integration can be straightforward. Second, large packages may have a large size of the behavior sequence which can exceed the 512 token limit of BERT. However, based on our analysis of packages in Table ??, we identify a relatively modest proportion (13.6%) of packages that exceed the limit. We plan to split the behavior sequence into multiple segments to circumvent the limit while maintaining their sequential integrity. Third, the proposed approach encounters higher false positives in real-world setting. We foresee several challenges and opportunities that could help reduce false positive rates. On the one hand, we can leverage ensemble methods. Utilizing a combination of different detection tools could be advantageous. An ensemble approach, where decisions are based on the consensus among multiple tools, might improve accuracy and reduce false positive rates. On the other hand, we can utilize dynamic analysis during our detection. Incorporating dynamic analysis into the detection process could provide deeper insights into the behavior of packages by capturing detailed runtime behaviors, potentially reducing false positives by confirming suspicious activities through actual execution rather than static analysis. Nonetheless, with a large number of new packages being released daily, it remains an open question how to leverage the benefits of dynamic analysis while mitigating its computational and time costs.

5 Related Work

Malicious Package Detection. Recently, there has been an increasing number of OSS supply chain attacks that inject malicious code into packages, with package registries such as NPM and PyPI being the largest targets [Lakshmanan(2022),?]. Ohm et al. [Ohm et al.(2020b)] were the first to systematically investigate malicious packages. They collected 174 real-world malicious packages from NPM, PyPI and RubyGems. Using this dataset, they manually analyzed how malicious behavior was injected (e.g., via typosquatting) and triggered (e.g., on installation), what the objective (e.g., data exfiltration) of malicious behavior was, and whether obfuscation was employed. Similarly, Guo et al. [Wenbo et al.(2023)] conducted an empirical study to understand the characteristics of the malicious code lifecycle in the PyPI ecosystem. Various approaches have also been proposed to detect malicious packages, mostly in NPM and PyPI.

In the NPM ecosystem, Zahan et al. [Zahan et al.(2022)] identified six signals of security weakness in NPM supply chain, and proposed a rule-based malicious package detector that searches for keywords in install scripts. Garrett et al. [Garrett et al.(2019)] selected features about whether libraries that access the network, file system and operating system processes are used, whether code is evaluated at runtime, and whether new files, new dependencies and new hookup script entries are present, and adopted clustering to build a benign behavior model which is used to detect malicious NPM packages. Differently, Ohm et al. [Ohm et al.(2020a)] adopted AST (abstract syntax tree)-level clustering to establish a malicious behavior model. Instead of using unsupervised methods, Fass et al. [Fass et al.(2018), ?] built a random forest classifier based on the frequency of specific patterns extracted from AST, CFG (control flow graph) and PDG (program dependency graph) of benign and malicious JavaScript samples. This approach is specifically designed for obfuscated JavaScript programs, and thus could be ineffective for non-obfuscated ones. Ohm et al. [Ohm et al.(2022)] extended the feature set of Garrett et al.’s work [Garrett et al.(2019)] by including features about package metadata and obfuscation. Similarly, Sejfia et al. [Sejfia and Schäfer(2022)] proposed AMALFI, which uses an extended feature set from Garrett et al.’s work [Garrett et al.(2019)] to train classifiers. These approaches capture malicious behavior as discrete features, hindering their accuracy in detecting malicious packages. Differently, we model malicious behavior in a sequential manner, which can model the maliciousness more precisely. Notice that Ferreira et al. [Ferreira et al.(2021)] proposed a lightweight permission system to protect applications from malicious package updates at runtime. Recently, Huang et al. [Huang et al.(2024)] proposes SpiderScan to identify malicious NPM packages based on graph-based behavior modeling and matching.

In the PyPI ecosystem, Vu et al. [Vu et al.(2020b)] proposed a rule-based approach to detect malicious PyPI packages that are spread by typosquatting and combosquatting attacks. The rule checks whether the package has the name similar to a Python standard library or a package with known source code repository. Besides, there are several other rule-based detectors, e.g., Bandit4Mal [Vu(2020)] and Malware Checks [Warehouse(2020)]. Vu et al. [Vu et al.(2023)] conducted a comprehensive evaluation of these detectors. Instead of relying on rules, Vu et al. [Vu et al.(2020a), ?] identified malicious packages based on discrepancies between the source code repository and distributed artifact of a Python package. Liang et al. [Liang et al.(2021)] used both package metadata features and code features to learn an isolated forest model for detecting malicious packages. Recently, Liang et al. [Wentao et al.(2023)] utilized clustering techniques to group the PyPI installation scripts so that outliers can be identified. Sun et al. [Sun et al.(2024)] proposes to integrate deep code behavior features with metadata features to detect malicious PyPI packages. These approaches share the same limitation with those detectors in NPM because they also model malicious behavior as discrete features.

Moreover, some approaches support different ecosystems. OSS Detect Backdoor [Microsoft(2020)] is a rule-based detector that support 15 ecosystems.

Taylor et al. [Taylor et al.(2020)] leveraged package name similarity and package popularity to detect malicious packages caused by typosquatting in NPM, PyPI and RubyGems. Ladisa et al. [Ladisa et al.(2023b)] utilized language-independent feature (e.g. number of URLs, entropy of strings in code files, etc.) to detect malicious packages in NPM and PyPI. However, these approaches are not based on high-level semantics of source code, which can produce high false positives. Different from the simple rule in Taylor et al. and Ladisa et al., Duan et al. [Duan et al.(2020)] derived five metadata analysis rules, four static analysis rules and four dynamic analysis rules to detect malicious packages in NPM, PyPI and RubyGems. However, it is heavyweight as it relies on program analysis.

Squatting attacks are also common in other domains, e.g., domain name system [Szurdi et al.(2014),?,?,?], container registry [Liu et al.(2022b)] and mobile app [Hu et al.(2020)]. Malware detection has been widely studied [Ye et al.(2017), Wenbo et al.(2023)]. OSS opens up new opportunities for malware detection due to the availability of source code.

Security Threats in OSS Supply Chain. Apart from malicious packages, there exist various security threats in OSS supply chain. Several studies [Cappos et al.(2008),?,?,?] have been conducted to investigate security threats in package registries such as NPM and PyPI. Ladisa et al. [Ladisa et al.(2023a)] reported a taxonomy of attacks on all OSS supply chain stages from code contributions to package distribution. They also assessed the safeguards against OSS supply chain attacks. Koishybayev et al. [Koishybayev et al.(2022)] and Gu et al. [Gu et al.(2023)] characterized security threats in continuous integration workflows that produce OSS.

Enck and Williams [Enck and Williams(2022)] summarized five challenges in OSS supply chain security, with the top one being updating vulnerable dependencies. Specifically, the vulnerability impact analysis along the supply chain has been widely explored in various ecosystems, e.g., NPM [Zerouali et al.(2022),?], Maven [Prana et al.(2021),?], PyPI [Ruohonen et al.(2021),?] and RubyGems [Zerouali et al.(2022),?]. To cope with vulnerable dependencies, various software composition analysis tools have been proposed [Xu et al.(2022),?,?,?]. Besides, several advances have been made on detect maliciousness in different stages of software development. Goyal et al. [Goyal et al.(2018)] and Gonzalez et al. [Gonzalez et al.(2021)] proposed to detect malicious commits on GitHub. Cao and Dolan-Gavitt [Cao and Dolan-Gavitt(2022)] tried to identify malware in GitHub forks. Lamb and Zacchiroli [Lamb and Zacchiroli(2021)] and Wheeler [Wheeler(2005)] attempted to avoid the injection of maliciousness in compilation by ensuring reproducible builds.

6 Conclusion and Future Work

xxxx [Cappos et al.(2008)]

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62332005, 62372114 and 62402342).

References

- [Agten et al.(2015)] Pieter Agten, Wouter Joosen, Frank Piessens, and Nick Nikiforakis. 2015. Seven months’ worth of mistakes: A longitudinal study of typosquatting abuse. In *Proceedings of the 22nd Network and Distributed System Security Symposium*.
- [Alfadel et al.(2023)] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. 2023. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering* 28 (2023).
- [Bagmar et al.(2021)] Aadesh Bagmar, Josiah Wedgwood, Dave Levin, and Jim Purtilo. 2021. I know what you imported last summer: A study of security threats in the python ecosystem. *arXiv preprint arXiv:2102.06301* (2021).
- [Cao and Dolan-Gavitt(2022)] Alan Cao and Brendan Dolan-Gavitt. 2022. What the Fork? Finding and Analyzing Malware in GitHub Forks. In *Proceedings of the Workshop on Measurements, Attacks, and Defenses for the Web*.
- [Cappos et al.(2008)] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. 2008. A look in the mirror: Attacks on package managers. In *Proceedings of the 15th ACM conference on Computer and communications security*. 565–574.
- [Duan et al.(2020)] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Towards measuring supply chain attacks on package managers for interpreted languages. In *Proceedings of 27th Annual Network and Distributed System Security Symposium*.
- [Enck and Williams(2022)] William Enck and Laurie Williams. 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100.
- [Fass et al.(2019)] Aurore Fass, Michael Backes, and Ben Stock. 2019. Jstap: A static pre-filter for malicious javascript detection. In *Proceedings of the 35th Annual Computer Security Applications Conference*. 257–269.
- [Fass et al.(2018)] Aurore Fass, Robert P Krawczyk, Michael Backes, and Ben Stock. 2018. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Proceedings of the 15th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 303–325.
- [Ferreira et al.(2021)] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing malicious package updates in npm with a lightweight permission system. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering*. 1334–1346.
- [Garrett et al.(2019)] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results*. 13–16.
- [Gkortzis et al.(2021)] Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. 2021. Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. *Journal of Systems and Software* 172 (2021).
- [Gonzalez et al.(2021)] Danielle Gonzalez, Thomas Zimmermann, Patrice Godefroid, and Max Schäfer. 2021. Anomalous: Automated detection of anomalous and potentially malicious commits on github. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice*. 258–267.
- [Goyal et al.(2018)] Raman Goyal, Gabriel Ferreira, Christian Kästner, and James Herbsleb. 2018. Identifying unusual commits on GitHub. *Journal of Software: Evolution and Process* 30, 1 (2018).

-
- [Gu et al.(2023)] Yacong Gu, Lingyun Ying, HuaJun Chai, Chu Qiao, Haixin Duan, and Xing Gao. 2023. Continuous Intrusion: Characterizing the Security of Continuous Integration Services. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1561–1577.
- [Gu et al.(2022)] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, HuaJun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2022. Investigating Package Related Security Threats in Software Registries. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1151–1168.
- [Hu et al.(2020)] Yangyu Hu, Haoyu Wang, Ren He, Li Li, Gareth Tyson, Ignacio Castro, Yao Guo, Lei Wu, and Guoai Xu. 2020. Mobile app squatting. In *Proceedings of The Web Conference 2020*. 1727–1738.
- [Huang et al.(2022)] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.
- [Huang et al.(2024)] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [Kaplan and Qian(2021)] Berkay Kaplan and Jingyu Qian. 2021. A survey on common threats in npm and pypi registries. In *Proceedings of the Second International Workshop on Deployable Machine Learning for Security Defense*. 132–156.
- [Khan et al.(2015)] Mohammad Taha Khan, Xiang Huo, Zhou Li, and Chris Kanich. 2015. Every second counts: Quantifying the negative externalities of cybercrime via typosquatting. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*. 135–150.
- [Kintis et al.(2017)] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. 2017. Hiding in plain sight: A longitudinal study of combosquatting abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 569–586.
- [Koishybayev et al.(2022)] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of Github {CI} Workflows. In *Proceedings of the 31st USENIX Security Symposium*. 2747–2763.
- [Ladisa et al.(2023a)] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023a. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1509–1526.
- [Ladisa et al.(2023b)] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023b. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 71–82.
- [Lakshmanan(2022)] Ravie Lakshmanan. 2022. Malware Strains Targeting Python and JavaScript Developers Through Official Repositories. Retrieved May 30, 2023 from <https://thehackernews.com/2022/12/malware-strains-targeting-python-and.html>
- [Lamb and Zacchiroli(2021)] Chris Lamb and Stefano Zacchiroli. 2021. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software* 39, 2 (2021), 62–70.
- [Liang et al.(2021)] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications*. 606–613.
- [Liu et al.(2022a)] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022a. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. 672–684.
- [Liu et al.(2022b)] Guannan Liu, Xing Gao, Haining Wang, and Kun Sun. 2022b. Exploring the Uncharted Space of Container Registry Typosquatting. In *Proceedings of the 31st USENIX Security Symposium*. 35–51.
- [Microsoft(2020)] Microsoft. 2020. OSS Detect Backdoor. Retrieved May 30, 2023 from <https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor>

- [Muncaster(2023)] Phil Muncaster. 2023. Hundreds Malicious Packages NPM. Retrieved May 30, 2023 from <https://www.infosecurity-magazine.com/news/hundreds-malicious-packages-npm/>
- [Ohm et al.(2022)] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the feasibility of supervised machine learning for the detection of malicious software packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–10.
- [Ohm et al.(2020a)] Marc Ohm, Lukas Kempf, Felix Boes, and Michael Meier. 2020a. Supporting the detection of software supply chain attacks through unsupervised signature generation. *arXiv preprint arXiv:2011.02235* (2020).
- [Ohm et al.(2020b)] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020b. Backstabber’s knife collection: A review of open source software supply chain attacks. In *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 23–43.
- [Ponta et al.(2018)] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution*. 449–460.
- [Prana et al.(2021)] Gede Artha Azriadi Prana, Abhishek Sharma, Lwin Khin Shar, Darius Foo, Andrew E Santosa, Asankhaya Sharma, and David Lo. 2021. Out of sight, out of mind? How vulnerable dependencies affect open-source projects. *Empirical Software Engineering* 26 (2021).
- [Ruohonen et al.(2021)] Jukka Ruohonen, Kalle Hjerppe, and Kalle Rindell. 2021. A Large-Scale Security-Oriented Static Analysis of Python Packages in PyPI. In *Proceedings of the 18th International Conference on Privacy, Security and Trust*. 1–10.
- [Sejfić and Schäfer(2022)] Adriana Sejfić and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering*. 1681–1692.
- [Sun et al.(2024)] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1₂: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [Szurdi et al.(2014)] Janos Szurdi, Balazs Kocso, Gabor Cseh, Jonathan Spring, Mark Felegyhazi, and Chris Kanich. 2014. The long “taile” of typosquatting domain names. In *Proceedings of the 23rd USENIX Security Symposium*. 191–206.
- [Tahir et al.(2018)] Rashid Tahir, Ali Raza, Faizan Ahmad, Jehangir Kazi, Fareed Zaffar, Chris Kanich, and Matthew Caesar. 2018. It’s all in the name: Why some URLs are more vulnerable to typosquatting. In *Proceedings of the 2018 IEEE Conference on Computer Communications*. 2618–2626.
- [Taylor et al.(2020)] Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Proceedings of the 14th International Conference on Network and System Security*. 112–131.
- [Vu(2020)] Duc-Ly Vu. 2020. Bandit4Mal. Retrieved May 30, 2023 from <https://github.com/lyvd/bandit4mal>
- [Vu et al.(2021)] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. Lastpymile: identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 780–792.
- [Vu et al.(2023)] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *Proceedings of the 45th International Conference on Software Engineering*.
- [Vu et al.(2020a)] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020a. Towards using source code repositories to identify software supply chain attacks. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 2093–2095.
- [Vu et al.(2020b)] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020b. Typosquatting and combosquatting attacks on the python ecosystem. In *Proceedings of the 2020 IEEE European Symposium on Security and Privacy Workshops*. 509–514.

-
- [Warehouse(2020)] Warehouse. 2020. Malware Checks. Retrieved May 1, 2023 from <https://warehouse.pypa.io/development/malware-checks.html>
- [Wenbo et al.(2023)] Guo Wenbo, Xu Zhengzi, Liu Chengwei, Huang Cheng, Fang Yong, and Liu Yang. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.
- [Wentao et al.(2023)] Liang Wentao, Ling Xiang, Wu Jingzheng, Luo Tianyue, and Yanjun Wu. 2023. A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.
- [Wheeler(2005)] David A Wheeler. 2005. Countering trusting trust through diverse double-compiling. In *Proceedings of the 21st Annual Computer Security Applications Conference*. 33–48.
- [Xu et al.(2022)] Meiqiu Xu, Ying Wang, Shing-Chi Cheung, Hai Yu, and Zhiliang Zhu. 2022. Insight: Exploring Cross-Ecosystem Vulnerability Impacts. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [Ye et al.(2017)] Yanfang Ye, Tao Li, Donald Adjeroh, and S Sitharama Iyengar. 2017. A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)* 50, 3 (2017), 1–40.
- [Zahan et al.(2022)] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.
- [Zerouali et al.(2022)] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. 2022. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering* 27 (2022).
- [Zimmermann et al.(2019)] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Security Symposium*. 995–1010.